

Chapter 16: Greedy Algorithms

1) Algorithms for optimisation problems go through a sequence of steps, with a set of choices at each step. For many optimisation problems, using dynamic programming to determine the best choices is overkill; simpler, more efficient algorithms will do. A greedy algorithm always makes a choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice will lead to globally optimal solution.

This heuristic strategy does not always produce an optimal solution, but sometimes it does.

2) Greedy algorithms are designed according to the following sequence of steps:-

- 1) Cast the optimisation problem as one in which we make a choice and are left with one subproblem to solve.
- 2) Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the greedy choice is always safe.
- 3) Demonstrate that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to

the original problem.

3) Beneath every greedy algorithm, there is almost always a more cumbersome dynamic-programming solution. Any problem to which greedy strategies apply, in general exhibit the greedy-choice property and optimal substructure.

Greedy Choice Property:

The greedy-choice property is that a globally optimal solution can be arrived at by making a locally optimal (greedy) choice.

In a greedy algorithm, we make whatever choice seems best at the moment and then solve the subproblem arising after the choice is made.

The choice made by a greedy algorithm may depend upon the choices made so far, but it cannot depend upon the future choices or on the solutions to subproblems.

In contrast, the choices made by a dynamic programming algorithm at every step, usually depends on the solutions to subproblems. Consequently, we typically solve dynamic-programming problems in a bottom-up manner, progressing from smaller subproblems to larger subproblems. But a greedy strategy progresses in a top down fashion, making one greedy choice after another, reducing each given problem

instance to a smaller one.

Optimal Substructure:

We use a very direct approach regarding optimal substructure, when applying it to greedy algorithms. By making a greedy choice in the original problem you arrive at a subproblem. All you need to do is argue that an optimal solution to the subproblem, combined with the greedy choice already made, yields an optimal solution to the original problem.

This scheme implicitly uses induction on the subproblems to prove that making the greedy choice at every step produces an optimal solution.

4) Greedy versus Dynamic Programming:

Because the optimal substructure property is exploited by both greedy and dynamic programming strategies, one might be tempted to generate a dynamic-programming solution to a problem when a greedy solution suffices, or one might ~~commonly~~ mistakenly think that a greedy solution works when in fact a dynamic programming solution is required.

For example, consider 2 variants of a classical optimization problem:-

1) **The 0-1 Knapsack problem:** A thief robbing a store finds n items. The i th item is worth v_i .

dollars and weights w_i pounds. (v_i and w_i are integers).

The maximum capacity of his knapsack is W pounds and he wants to take as valuable a load as possible. This is called a 0-1 knapsack problem because each item must either be taken or left behind.

2) The fractional knapsack problem:

The thief can now take fractions of items rather than having to make a binary choice (0-1) for each item.

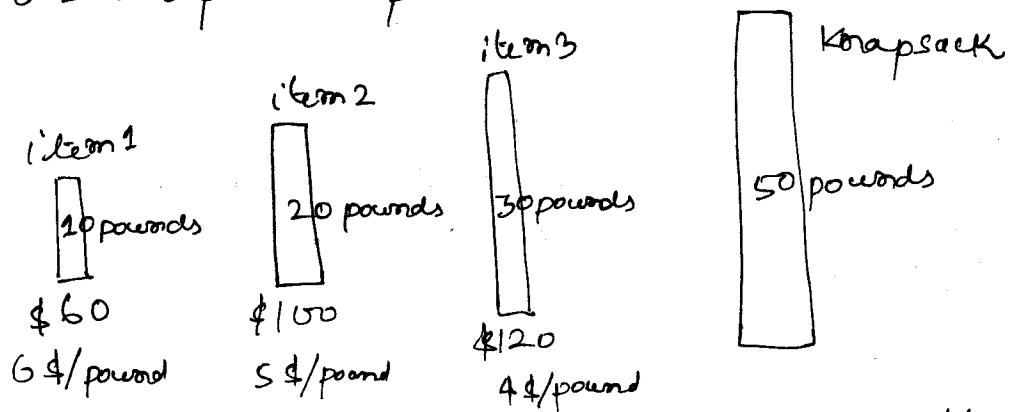
→ Both knapsack problems exhibit the optimal-substructure property. For the 0-1 problem consider the most valuable load that can weight at most W pounds. If we remove this item weighing w_j , the remaining load must be the most valuable load weighing at most $W - w_j$ which the thief can take from $n-1$ original items. For the comparable fractional problem if we remove a weight w of one item j from the optimal load the remaining load must be the most valuable load weighing at most $W - w$ that the thief can take from the original $n-1$ items plus $w_j - w$ pounds of item j .

→ The fractional knapsack problem is solvable by a greedy strategy, whereas the 0-1 problem is not. To solve the fractional problem we first compute the value per

pound i.e. V_i/W_i for every item. The thief begins by taking as much as possible of the item with the greatest value per pound. If the supply of that item is exhausted and he can still carry more, he takes as much as possible of the item with the next greatest value per pound, and so forth till he can't carry any more.

This greedy algorithm runs in $O(n \log n)$ time ~~provided~~ by sorting the items by value-per-pound.

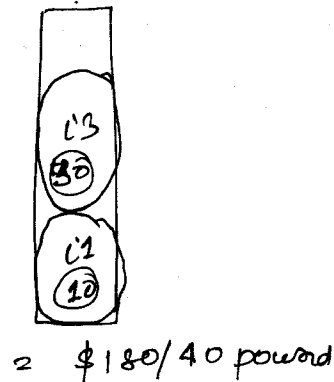
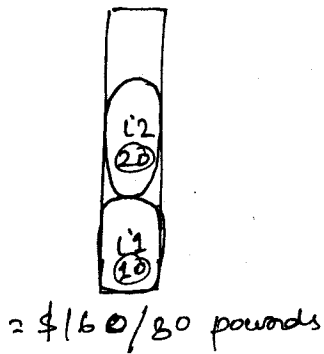
This greedy strategy does not work for the 0-1 knapsack problem. For eg: consider 3 items.



Greedy strategy would take item 1 first.

Greedy solution:-

Alternate solution using item 1



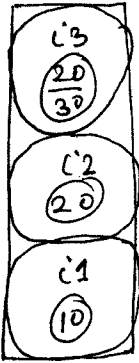
Dynamic programming solution:-



Clearly, the greedy solution to the 0/1 knapsack problem is non-optimal.

= \$220/50 pounds

greedy solution for a ^{comparable fractional} knapsack problem:-



\$240/50 pounds

This solution is indeed optimal and could also have been arrived using a more cumbersome dynamic programming approach.

5) Huffman Codes:

→ Any file of information comprising of a sequence of characters can be represented using a "binary character code" wherein each character is represented by a unique binary string.

2 kinds of binary character codes exist:-

- (i) Fixed-length codes: here every character is granted the same no. of bits
- (ii) Variable-length codes: characters occurring with large frequencies are given short codewords and infrequent characters are given long codewords.

→ **Prefix Codes:** A code in which no codeword is also a prefix of some other codeword is called a prefix code. There is no loss of generality in restricting attention to prefix codes, because any optimal data compression achievable by some character code can always be achieved with a prefix code.

Prefix codes are desirable because they simplify decoding. Since no codeword is a prefix of any other, the codeword that begins an encoded file is unambiguous. We can simply identify the initial codeword, translate it back to the original character, and repeat the decoding process on the remainder of the encoded file.

→ **Eg:**

	a	b	c	d	e	f
Frequency ($\times 10^5$)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length "	0	101	100	111	1101	1100

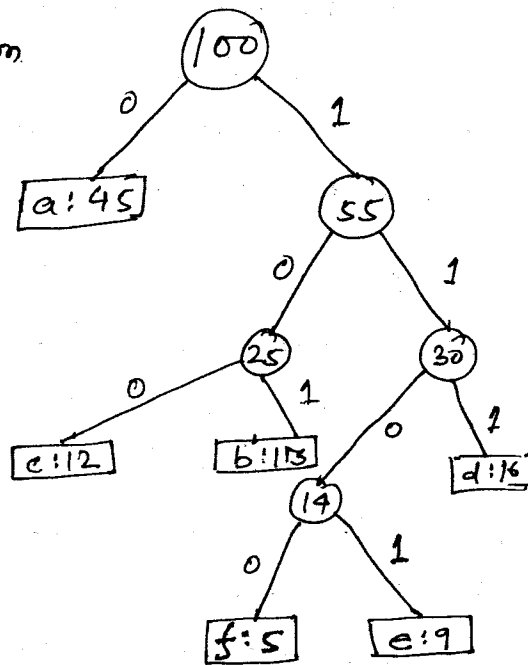
Encoding is a simple table lookup procedure:-

$$abc = 0 \cdot 101 \cdot 100 = 010100$$

→ Decoding needs a convenient representation for the prefix code so that the initial codeword can be easily picked off.

An optimal prefix code for a file is represented by a full binary tree. The characters of the alphabet are represented by the leaves of the binary tree.

If C be an alphabet from which characters are drawn, then a full binary tree for an optimal prefix code has exactly $|C|$ leaves and exactly $|C|-1$ internal nodes.



→ The cost of a full binary tree representing an optimal prefix code is defined to be the number of bits required to encode a file.

For every character c in the alphabet C let:-

$f(c)$ = frequency of occurrence of c in the file

$d_T(c)$ = depth of the leaf representing the character c in the Tree T = length of the codeword for c

c in the Tree T = length of the codeword for c

$$\text{cost}(T) = B(T) = \sum_{c \in C} f(c) d_T(c)$$

no. of bits required to encode the file

→ Huffman Code:

Huffman invented a greedy algorithm for constructing an optimal prefix code called a Huffman code.

The procedure HUFFMAN takes as input a set C of n characters. It ~~takes~~ builds the optimal tree T corresponding to the optimal prefix code in a bottom-up manner. It begins with a set of $|C|$ leaves and performs a sequence of $|C|-1$ merging operations to create the final tree.

A min-priority queue Q , keyed on f , is used to identify the two least-frequent objects to merge together.

HUFFMAN(C)

```
1   $n \leftarrow |C|$ 
2   $Q \leftarrow C$ 
3  for  $i \leftarrow 1$  to  $n-1$ 
4      do allocate a new node  $z$ 
5           $\text{left}[z] \leftarrow x \leftarrow \text{EXTRACT-MIN}(Q)$ 
6           $\text{right}[z] \leftarrow y \leftarrow \text{EXTRACT-MIN}(Q)$ 
7           $f[z] \leftarrow f[x] + f[y]$ 
8           $\text{INSERT}(Q, z)$ 
9  return  $\text{EXTRACT-MIN}(Q)$  ▷ Return the root of the tree
```

~~The~~ x is made the left child of Z and y is made the right child of Z .
Switching the left and right child of any node yields a different code of the same cost.

The for loop executes exactly $(n-1)$ times and since each heap operation requires time $O(\log n)$ the procedure runs in time $O(n \log n)$.

→ Correctness of Huffman's algorithm:

To prove that the greedy algorithm of Huffman is correct, we show that the problem of determining an optimal prefix code exhibits the greedy choice and optimal substructure properties.

Lemma:

Let C be an alphabet and x, y be 2 characters in C having the lowest frequencies. Then there exists an optimal prefix code for C in which the codewords for x and y have the same length and differ only in the last bit.

If you view the cost as a single merger as the sum of the frequencies of the 2 items being merged. The above lemma implies that the procedure of building an optimal tree, can begin with the greedy choice of merging together

those 2 characters of lowest frequency. HUFFMAN
always chooses the merger that incurs the least
cost.